

ExoSync / SysEng

Procédure de Validation Manuelle

Cockpits Ingénieur & Dashboard Métier

Date : 17/06/2026
Version : 1.0 — premier test end-to-end
Projet : SysEng Alstom — ExoSync MVP
Auteur : Jean-Luc Chaudon

Sommaire

#	Section	Contenu
1	Pré-requis	Dépendances Python, tests de non-régression
2	Test automatisé (script demo)	Cycle complet en une commande
3	Validation cockpit ingénieur	Vérifications onglet par onglet dans Excel
4	Validation dashboard métier	KPIs, alertes, filtrage, _Manifeste
5	Simulation de saisie réelle	Modifier les données et vérifier le cycle push/pull
6	Bilan et prochaines étapes	État actuel, gaps CLI, registre.json

1 — Pré-requis

Avant de lancer les tests, s'assurer que l'environnement est correct :

1.1 Dépendances Python

Depuis la racine du projet **SysEng** :

```
pip install -r requirements.txt
```

Librairies clés utilisées : **openpyxl**, **pydantic**. La génération Excel ne nécessite pas LibreOffice.

1.2 Suite de tests — non-régression

Lancer la suite complète avant tout test manuel :

```
python -m pytest -q
```

Résultat attendu : **tous les tests passent**, zéro échec, zéro erreur.

Note : Si des tests échouent avant même de commencer les validations manuelles, ne pas continuer — corriger d'abord la régression.

1.3 Fichiers de configuration requis

Check	Résultat attendu
config/uo_instances.json	Instances UO réelles du projet (5 UOs minimum)
config/acteurs.json	Profils acteurs — doit contenir Jean-Luc Bernard (USR004)
output/cockpits/	Dossier créé automatiquement par le script
output/store.json	Créé/mis à jour par le script (données simulées)

2 — Test automatisé (script demo)

Le script **scripts/demo_cockpits.py** orchestre le cycle complet en une seule commande :

```
python scripts/demo_cockpits.py
```

2.1 Ce que fait le script

Étape	Action
Étape 1	Charge les UOs et acteurs réels depuis config/
Étape 2	Génère les 3 cockpits ingénieurs dans output/cockpits/
Étape 3	Injecte 10 valeurs simulées dans output/store.json (simule la saisie Excel + sync ExoSync)
Étape 4	Génère Dashboard_Jean-Luc_Bernard.xlsx
Étape 5	Vérifie automatiquement 4 points critiques et affiche ■ / ■

2.2 Sortie attendue (4 vérifications)

```
Filtrage (Denis Renard absent) : ■
UO-001 (Alice) présente       : ■
Alerte dérive UO-002 (52h>48h) : ■
Avancement UO-001 = 65%       : ■
```

BLOQUANT

Si une vérification affiche ■, arrêter — le générateur est en erreur.

2.3 Fichiers générés

Fichier	Taille	Contenu
Cockpit_Alice_Dubois.xlsx	~9 Ko	Cockpit d'Alice (UO-001, UO-002)
Cockpit_Bruno_Lecomte.xlsx	~9 Ko	Cockpit de Bruno (UO-003, UO-004)
Cockpit_Camille_Vidal.xlsx	~9 Ko	Cockpit de Camille (UO-005)
Dashboard_Jean-Luc_Bernard.xlsx	~11 Ko	Dashboard consolidé équipe

3 — Validation cockpit ingénieur

Ouvrir **output/cockpits/Cockpit_Alice_Dubois.xlsx** dans Excel. Vérifier les 3 onglets ci-dessous.

3.1 Onglet « Mes UOs »

Check	Résultat attendu
Ligne 5	Headers : UO ID Type UO Système Projet Charge (h) % Avancement H réalisées Date fin A
Lignes 6+	Une ligne par UO — UO-001 et UO-002 présentes (pas UO-003)
Col F (% Avancement)	Fond JAUNE — zone de saisie ingénieur
Col G (H réalisées)	Fond JAUNE — zone de saisie ingénieur
Col I (Alerte)	Formule Excel visible : =IF(G6>E6,"■ Dérive heures",...) — pas de fond jaune
Charge allouée	Valeur numérique dans col E (32h pour UO-001, 48h pour UO-002)

3.2 Onglet « Agenda »

Check	Résultat attendu
Structure	3 sections : « Semaine en cours », « Prochaines échéances », « Points ouverts / Actions »
En-têtes de section	Fond bleu foncé avec titre en blanc
Colonnes	UO ID Activité Priorité Date échéance Statut Action
Zone saisie Agenda	Colonne « Action » en fond jaune (saisie libre)
Points ouverts	Lignes vides jaunes — zone à remplir par l'ingénieur

3.3 Onglet « _Manifeste »

Check	Résultat attendu
Ligne 1	Cellule A1 = MANIFESTE_V=1
Ligne 2	12 headers : TYPE SCOPE NOM_GLOBAL NOM_LOCAL FEUILLE TABLEAU CLE COLONN
Règles PUSH	2 règles par UO : avancement (col F) et heures_realisees (col G) vers store
Règles PULL	2 règles par UO : charge_allouee et date_fin depuis store
COMMENTAIRE	Colonne 12 remplie — description en français pour chaque règle (>10 chars)
Séparation	Ligne vide entre les règles de chaque UO

4 — Validation dashboard métier

Ouvrir **output/cockpits/Dashboard_Jean-Luc_Bernard.xlsx**. Le dashboard consolide les 5 UOs de l'équipe (Alice + Bruno + Camille).

4.1 Onglet « Synthèse »

Check	Résultat attendu
KPIs (ligne 2)	Nb UOs = 5, Charge totale = 192h, % moyen = environ 0.5, Nb alertes = 2
Tableau consolidé	5 lignes de données à partir de la ligne 6
Filtrage	Denis Renard ABSENT — l'acteur USR004 filtre par ingénieur

Check	Résultat attendu
Colonne Ingénieur	Nom de l'ingénieur visible sur chaque ligne
Colonne Alerte	UO-002 : « Dérive heures » — UO-003 : potentiellement « Échéance proche »

4.2 Onglet « Par Ingénieur »

Check	Résultat attendu
Structure	Une section par ingénieur : Alice (2 UOs), Bruno (2 UOs), Camille (1 UO)
En-têtes	Fond bleu avec nom ingénieur + total charge + % avancement moyen
Colonnes UO	UO ID Type Système Projet Charge % Avancement H réalisées Date fin
Lien cockpit	Cellule « Voir cockpit » avec lien hypertexte vers fichier .xlsx

4.3 Onglet « Alertes »

Check	Résultat attendu
UO-002	Présente en alerte Critique (fond ROUGE) — 52h réalisées > 48h allouées
Tri criticité	Alertes Critique avant Élevée
Colonnes	Ingénieur UO ID Type alerte Détail Criticité
Pas d'alerte	Si aucune alerte : bandeau vert « Aucune alerte active »

4.4 Onglet « _Manifeste »

Check	Résultat attendu
Ligne 1	MANIFESTE_V=1
Règles	3 règles PULL par UO : avancement, heures_realisees, points_ouverts
Nombre lignes	5 UOs × 3 règles = 15 règles de données (+ lignes vides entre UOs)
COMMENTAIRE	Colonne 12 remplie pour chaque règle

5 — Simulation de saisie réelle

Pour aller plus loin, modifier les données simulées et relancer le script pour vérifier que le dashboard s'adapte.

5.1 Modifier les saisies dans le script

Éditer **scripts/demo_cockpits.py**, section **SAISIES_SIMULEES** (ligne ~41) :

```
SAISIES_SIMULEES = {
    # Passer UO-001 en dérive : 38h > 32h allouées
    "uo.UO-001.avancement": 0.90,
    "uo.UO-001.heures_realisees": 38.0,    # ← dérive
    # ...
}
```

Relancer ensuite :

```
python scripts/demo_cockpits.py
```

L'alerte doit apparaître sur UO-001 dans l'onglet Alertes du dashboard.

5.2 Scénarios de test recommandés

Scénario	Action	Résultat attendu
Dérive heures	heures_realisees > charge_allouee pour une UO	Ligne rouge dans Alertes, alerte dans col I du cockpit
Échéance proche	date_fin < aujourd'hui + 7j (modifier config)	Alerte « Échéance proche » dans Alertes
Avancement 0%	avancement = 0.0 sur une UO à date passée	Potentielle alerte retard dans Alertes
Tous OK	Toutes les heures sous le seuil, avancements complétés	Complète Alertes affiche le bandeau vert

6 — Bilan et prochaines étapes

État du pipeline à l'issue de cette première vague de tests :

Composant	Fichier / Commande	Statut
Suite de tests (361 tests)	pytest -q	■ PASS
Génération cockpit ingénieur	cockpit_ingénieur_generator.py	■ OK
Génération dashboard métier	dashboard_metier_generator.py	■ OK
Cycle push/pull automatisé	scripts/demo_cockpits.py	■ OK
Validation visuelle Excel	output/cockpits/*.xlsx	■ Manuel
CLI generate-cockpit	main.py → cockpit_generator.py (ancien)	■ Non branché
Registre.json	Nouveaux fichiers non déclarés	■ Non mis à jour

Prochaines étapes (CONV-08)

- Brancher **cockpit_ingénieur_generator** dans **main.py** (nouveau sous-commande *generate-cockpit-v2* ou remplacement de l'ancien)
- Enregistrer les nouveaux fichiers *Cockpit_*.xlsx* dans **config/registre.json** pour que *python main.py sync* les traite
- Test end-to-end complet : saisie manuelle dans Excel → sync → vérification dashboard
- Vérifier la confidentialité du repo GitHub (*L09U1-CFL2400-CLIM.xlsx* peut avoir été poussé)